DSCI 551 Final Report

Meowlytics: PostgreSQL B-tree Indexing, Query Planning, and MVCC for an AI-Powered Cat Food Ingredient Analyzer

Student: Wei Xing
Course: DSCI 551
Database System: PostgreSQL
Project Repository: https://github.com/weixingwork/dsci551-project-meowlytics.git
Google Drive (code + docs):

https://drive.google.com/drive/folders/1dWIb3oZXquhc0hyHdwrQEqxDGPO5qVd_?usp=sharing

---

1. Introduction & Motivation

Cat-food ingredient lists are long and technical (e.g., *Taurine*, *Mixed Tocopherols*, *Animal By-Product Meal*), and the same component is often labeled inconsistently across brands.

Meowlytics is a web app that takes a photo of a cat-food label, extracts the ingredient list with Gemini 2.5 Flash, and enriches each ingredient from a PostgreSQL knowledge base. Users can save analyses to a personal *Favorites* collection.

This report focuses on three PostgreSQL internals and their mapping to application operations:

1. **B-tree indexing** — exact lookup and composite filter+sort retrieval.
2. **Cost-based query planning** — when an index is used vs. a sequential scan.
3. **MVCC** — concurrent reads and writes during AI-driven inserts.

All claims are backed by `EXPLAIN ANALYZE` scripts in `dsci551/explain/` against a deterministic seed of 10,000 ingredients, 51 users, and 5,000 favorites.

---

2. Database System Overview

PostgreSQL is an open-source object-relational database with ACID guarantees, a cost-based planner, multiple index types (B-tree, Hash, GiST, GIN, BRIN), and a transparent execution model via `EXPLAIN ANALYZE`.

Reasons for choosing it:

- **Transparent internals.** `EXPLAIN (ANALYZE, BUFFERS, VERBOSE)` exposes every plan decision, which is essential for connecting application behavior to internals.
- **B-tree fit.** Exact lookups (`WHERE nameEn = 'Chicken'`) and user-scoped ordered retrieval (`WHERE userId = ? ORDER BY createdAt DESC`) map cleanly onto B-tree access paths.
- **MVCC.** Read-heavy traffic interleaves with bursty AI-driven writes; MVCC lets readers see a consistent snapshot without blocking writers.
- **Heap + secondary indexes** avoid pre-committing to a single physical sort order, unlike clustered storage (InnoDB) or document storage (MongoDB).

The app uses **Prisma 7** with the `@prisma/adapter-pg` driver. Pooling uses `pg.Pool` (see `lib/db.ts`).

---

3. Internal Architecture

3.1 Heap Storage

Tables are stored as fixed-size **8 KB heap pages**. Rows are unordered tuples; physical position reflects insertion order plus MVCC relocations. Each tuple is addressed by a `(page, offset)` pair called **`ctid`**.

The four Meowlytics tables (`Ingredient`, `User`, `Favorite`, `Folder`) all use heap storage. `Ingredient` occupies ~1,167 pages for 10,000 rows (confirmed by `Buffers: shared hit` in the seq-scan plan).

3.2 B-tree Indexing

Indexes are **secondary structures**: an index entry is a `(key, ctid)` pair stored in a file separate from the heap. The default — and only — index type used here is the **B-tree**: a balanced tree whose leaf pages hold keys in sorted order.

Lookup is `O(log n)`: traverse root → leaf, then follow the `ctid` to the heap. The `Ingredient_nameEn_idx` lookup for `'Chicken'` touches **5 buffer pages** (3 index + 2 heap) vs. 1,167 for a seq scan.

Four B-tree indexes are declared (excluding implicit PK indexes), each tied to a query pattern:

Index	Table	Purpose
`Ingredient_name_idx`	`Ingredient`	Chinese-name lookup
`Ingredient_nameEn_idx`	`Ingredient`	English-name lookup (hot path)
`Ingredient_source_idx`	`Ingredient`	Filter by `knowledge_base` vs `ai_generated`
`Favorite_userId_createdAt_idx`	`Favorite`	Composite: filter by user + sort by time

The complete Prisma schema with all four models and their index declarations is in **Appendix C**.

3.3 Query Execution and the Cost-Based Planner

Queries go through **parse** → **rewrite** → **plan** → **execute**. The planner enumerates candidates (seq scan, index scan, index-only, bitmap heap, ...) and costs each using `pg_statistic` (refreshed by `ANALYZE`). The cheapest wins.

Default cost units: `seq_page_cost = 1.0`, `random_page_cost = 4.0`. So whether an index wins depends on **predicate selectivity** and **table size**, not just its existence — demonstrated empirically in Sec. 5.

3.4 MVCC and Tuple Versioning

PostgreSQL never updates in place. Each tuple has two hidden columns:

- **`xmin`** — `xid` that inserted this version.
- **`xmax`** — `xid` that deleted/updated it (0 if live).

An `UPDATE` writes a new tuple with a new `xmin`/`ctid` and stamps `xmax` on the old. Both versions coexist until **VACUUM** reclaims the dead one. Readers use a snapshot of visible `xid`s, so concurrent reads and writes don't take explicit read locks.

Consequences:

1. Reads don't block writes.
2. Long transactions accumulate dead tuples ("bloat").
3. Every `UPDATE` must also insert new index entries (new `ctid`).

3.5 Distribution

Out of scope (single-person project, allowed by guideline). Meowlytics runs on one node. PostgreSQL supports distribution via logical replication and Citus, but exploring these exceeds scope.

---

4. Application Design

4.1 Architecture

Meowlytics is built on **Next.js 16** (App Router). One Node.js process serves the React frontend and the JSON API under `app/api/`. PostgreSQL is reached through Prisma 7 via `.env`.

Layers:

- **UI** (`app/(main)/`): analysis, favorites, compare, account; React Server Components + Zustand.
- **API** (`app/api/`): auth, knowledge lookup, favorites CRUD, AI analysis.
- **Domain** (`lib/`): auth, sessions, knowledge search, validation.
- **DB**: Prisma client + raw SQL where needed (e.g., seed `ANALYZE`).

4.2 Data Model

Four tables (indexes in Sec. 3.2):

- **`Ingredient`** — 10,000 rows. Chinese/English names, aliases, category, health impact, description. `source` ∈ {`knowledge_base`, `ai_generated`}.
- **`User`** — 51 rows (1 demo + 50 synthetic). Unique email; scrypt salt+hash password.
- **`Favorite`** — 5,000 rows (2,000 for the demo user). `analysis` is a `Json` blob; FKs to `User` and optional `Folder`.
- **`Folder`** — empty in seed; used by the UI for grouping.

4.3 Application Workflow

1. **Image upload** via the analysis page.
2. **AI extraction** — Gemini 2.5 Flash returns a structured ingredient list.
3. **Knowledge enrichment** — `POST /api/knowledge` runs a lookup chain (exact name → English → alias → fuzzy fallback).
4. **AI fallback insert** — unknown ingredients are generated by Gemini and inserted with `source = 'ai_generated'` so subsequent users hit the DB.
5. **Save to favorites** — persists a `Favorite` row.
6. **Retrieve favorites** — single indexed query.

4.4 Reproducibility

`dsci551/setup.sh` creates the DB, adapts `.env`, runs `prisma db push`, seeds, and runs a smoke `EXPLAIN ANALYZE`. The seed uses a fixed-seed Mulberry32 PRNG, so the planner statistics — and every plan in this report — are reproducible.

---

5. Mapping Internals to Application Behavior

For each operation: (a) application behavior, (b) SQL, (c) internal behavior, (d) `EXPLAIN ANALYZE` evidence. Evidence files in `dsci551/explain/`.

5.1 Mapping 1 — Exact Ingredient Lookup → B-tree Index Scan

Application behavior. Clicking an ingredient (e.g. "Chicken") triggers `POST /api/knowledge`. `lib/knowledge/db-search.ts` runs:

```
SELECT * FROM "Ingredient" WHERE "nameEn" = 'Chicken' LIMIT 1;
```

Internal mechanism. The planner picks `Ingredient_nameEn_idx` for the equality predicate. Execution traverses root → leaf, finds the `(key, ctid)`, then fetches the heap tuple.

Evidence. Script `01-ingredient-exact-lookup.sql` produces:

```
Index Scan using "Ingredient_nameEn_idx" on "Ingredient"
  (cost=0.29..8.30 rows=1 width=428)
  (actual time=0.030..0.033 rows=1 loops=1)
  Index Cond: ("nameEn" = 'Chicken'::text)
  Buffers: shared hit=5
Execution Time: 0.069 ms
```

Why it matters. Only 5 of 1,167 pages touched. At 0.069 ms, the lookup scales logarithmically.

5.2 Mapping 2 — Favorites Page → Composite Index, Backward Scan

Application behavior. Opening `/favorites` triggers `app/api/favorites/route.ts`:

```
SELECT * FROM "Favorite" WHERE "userId" = $1 ORDER BY "createdAt" DESC;
```

Internal mechanism. The composite index `Favorite_userId_createdAt_idx` is physically sorted by `(userId, createdAt)`, so PostgreSQL descends to the first leaf for the target `userId` and reads leaf entries **backward** for `createdAt DESC`. The plan contains an `Index Scan Backward` and **no `Sort` node**.

Evidence. Script `02-favorites-composite-index.sql` produces:

The script adds `LIMIT 20` to keep the evidence output compact and to demonstrate page-sized retrieval. The application route currently retrieves all of the user's favorites in the same indexed order.

```
Limit  (cost=6.92..63.30 rows=20 width=1650) (actual time=0.067..0.145 rows=20 loops=1)
  Buffers: shared hit=28
  ->  Index Scan Backward using "Favorite_userId_createdAt_idx" on "Favorite"
        (cost=0.28..276.53 rows=98 width=1650)
        (actual time=0.066..0.142 rows=20 loops=1)
        Index Cond: ("userId" = $0)
Execution Time: 0.192 ms
```

Why it matters. A single-column index on `userId` would still need a sort; one on `createdAt` would force a filter pass. The composite index serves filter *and* sort in one pass.

5.3 Mapping 3 — Cost-Based Planner: Same Query, With vs Without Index

Application behavior. Script `04-with-vs-without-index.sql` runs the lookup, drops the index, reruns, recreates, reruns.

Internal mechanism. Without the index, the only plan is a seq scan of 10,000 rows. With it, index-scan cost (8.30) beats seq-scan cost (1292.00).

Evidence.

Plan	Cost	Execution Time	Buffers
With index	0.29..8.30	0.105 ms	5
Without index	0.00..1292.00	4.149 ms	1167

Why it matters. 40× faster execution, 233× fewer buffer fetches — empirical justification for the index. Having an index is not enough; the planner must also judge it cheaper.

5.4 Mapping 4 — Cost-Based Planner: Small Table vs Large Table

Application behavior. Script `05-small-vs-large-data.sql` runs the same lookup against a 100-row temp copy and the 10,000-row `Ingredient` (both indexed).

Internal mechanism. A 100-row heap fits in a few pages (seq-scan cost ~7.25), and the index has a startup cost of ~0.29 plus random-page costs — so the index loses on tiny tables. Planner picks `Seq Scan` at 100 rows, `Index Scan` at 10,000.

Evidence.

Table size	Chosen plan	Cost	Why
100 rows	Seq Scan	7.25	Index startup overhead not worth it
10,000 rows	Index Scan	8.30	Index path much cheaper than 1292

Why it matters. Index usage is statistic-driven, not rule-based. `ANALYZE` after data changes is essential.

5.5 Mapping 5 — B-tree Limitation: Leading-Wildcard Fuzzy Search

Application behavior. Fuzzy search uses `ILIKE '%chick%'`.

Internal mechanism. B-tree is sorted by prefix, so prefix queries (`'chick%'`) hit a contiguous leaf range. A leading wildcard (`'%chick%'`) has no contiguous range, so the planner falls back to seq scan + filter.

Evidence. Script `03-fuzzy-ilike-seq-scan.sql` produces:

```
Seq Scan on "Ingredient"  (cost=0.00..1292.00 rows=606)
  (actual time=0.138..11.892 rows=600 loops=1)
  Filter: ("nameEn" ~~* '%chick%'::text)
  Rows Removed by Filter: 9400
  Buffers: shared hit=1167
Execution Time: 11.970 ms
```

Why it matters. A real B-tree limitation. The fix is a `pg_trgm` GIN index on trigrams of `nameEn`; deliberately out of scope here, documented in Sec. 7.

5.6 Mapping 6 — AI Insert: Heap Insert + Index Maintenance

Application behavior. `saveAIGeneratedIngredient`:

```
INSERT INTO "Ingredient" (...) VALUES (...);
```

Internal mechanism.

1. Append tuple to a heap page.
2. Insert `(key, ctid)` into each of three B-trees (`name`, `nameEn`, `source`).

3. Split + propagate if a leaf is full.

`xmin` is set to the inserting `xid`, so the row is invisible to in-flight transactions with earlier snapshots.

Why it matters. Read-vs-write tradeoff. Seeding 10,000 rows with three indexes takes ~1.5 s — invisible at app level, but indexes aren't free.

5.7 Mapping 7 — MVCC: Concurrent Read While Writer Commits

Application behavior. A favorites `SELECT` runs while another request inserts or refreshes an ingredient. Reader must not block writer and must see a consistent snapshot.

Internal mechanism. MVCC via `xmin` / `xmax`. An `UPDATE`:

1. Writes a new tuple with new `xmin` / `ctid`.
2. Stamps `xmax` on the old version.

A reader's snapshot is fixed at transaction start, so it sees the old version until it commits.

Evidence. Script `06-mvcc-snapshot-isolation.sql` demonstrates tuple versioning in a single session:

Stage	`xmin`	`xmax`	`ctid`
Before UPDATE	1249	0	(100,18)
After UPDATE (in txn)	1277	0	(100,19)
After ROLLBACK	1249	1277	(100,18)

`n_dead_tup` increases by 1 after rollback — the new version was physically written and stays on disk until VACUUM. A two-session test also confirms that a `REPEATABLE READ` reader sees the old value even after a concurrent commit.

Why it matters. Reads stay resilient under write bursts. Cost: bloat until autovacuum runs.

---

6. Comparison with MySQL and MongoDB

Feature	PostgreSQL (this project)	MySQL (InnoDB)	MongoDB
Primary storage layout	Heap pages + secondary B-tree indexes. Rows are not physically ordered by any column.	**Clustered** B-tree on the primary key — rows are physically sorted by PK; secondary indexes store the PK, not a row pointer.	BSON documents stored in collections, each document self-contained.
Index data structure	B-tree (default), Hash, GiST, GIN, BRIN	B-tree, Hash, FULLTEXT (full-text), R-tree (spatial)	B-tree, hashed, text, geospatial
Concurrency model	MVCC with snapshot isolation; new tuple versions live in the heap until VACUUM.	MVCC, but old versions are kept in a separate **undo log** rather than in the data pages.	MVCC since WiredTiger; document-level locking.
Query planner	Cost-based, transparent via `EXPLAIN ANALYZE` (BUFFERS, VERBOSE).	Cost-based; `EXPLAIN` exists but historically less detailed than PostgreSQL.	Rule + cost hybrid; `explain()` returns winning + rejected plans.
Fuzzy search	Requires `pg_trgm` + GIN to index leading-wildcard `ILIKE`.	Native `FULLTEXT` indexes for word-level search.	Native text indexes with stemming and language analyzers.
Distribution	Single-node by default; logical replication and Citus extension for sharding.	Native replication, group replication, MySQL Cluster (NDB).	First-class horizontal sharding by shard key.
Best fit for this app	Transparent planner, mixed read/write, relational join model, snapshot isolation.	Reasonable fit but clustered storage would force a single PK ordering.	Schema flexibility appealing for AI-generated records, but loses transactional joins (User ↔ Favorite).

Why PostgreSQL specifically. At this scale, all three are fast enough. The deciding factors are transparency and fit:

- `EXPLAIN ANALYZE` makes every claim in Sec. 5 directly verifiable.
- InnoDB's clustered storage and undo-log MVCC are harder to demonstrate live than PostgreSQL's tuple-version MVCC.
- MongoDB suits flexible schemas, but the relational `User → Favorite` link and ordered retrieval favor SQL.

---

7. Limitations & Lessons Learned

7.1 Limitations

1. **Fuzzy search not indexed.** Leading-wildcard `ILIKE` forces a seq scan (Sec. 5.5). Fix: `pg_trgm` + GIN. Scoped out to keep index discussion focused on B-tree.
2. **Live AI requires `GOOGLE_API_KEY`.** Seed pre-populates 10,000 ingredients so Sec. 5 evidence runs without AI calls.
3. **MVCC write overhead.** Each `UPDATE` writes a new tuple and new index entries. Read-heavy workload masks bloat costs here.
4. **Single-node only.** Distribution not demonstrated (out of scope per guideline).
5. **In-memory fuzzy fallback.** `lib/knowledge/db-search.ts:130` loads the table into memory if indexed lookups

miss. OK at 10k rows; `pg_trgm` GIN would replace it.

7.2 Lessons Learned

1. **Indexes follow query patterns.** The composite `(userId, createdAt)` serves filter and sort in one pass — column-by-column design would have missed this.
2. **Cost-based, not rule-based.** Seq Scan on a 100-row indexed table (Sec. 5.4) is correct.
3. **`EXPLAIN ANALYZE` is ground truth** — buffer hits and actual timing, not estimates.
4. **Reproducibility = deterministic seed + one-command setup**, not optional.
5. **MVCC = "insert + tombstone."** Demystifies vacuum cost, bloat, and snapshot isolation.

---

8. Conclusion

Meowlytics maps PostgreSQL internals to seven application operations: exact lookup → Index Scan (Sec. 5.1); favorites → Composite Backward Index Scan (Sec. 5.2); cost model with vs without index (Sec. 5.3); plan flips with data size (Sec. 5.4); leading-wildcard ILIKE → Seq Scan (Sec. 5.5); insert → heap + 3 B-tree maintenance (Sec. 5.6); MVCC tuple versioning enables non-blocking reads (Sec. 5.7). Every mapping is backed by a reproducible `EXPLAIN ANALYZE` script and a deterministic 10k-row seed.

Appendix A: How to Reproduce

A.1 Prerequisites

- **Node.js** ≥ 20 and **npm** ≥ 10
- **PostgreSQL** ≥ 14 running locally on the default port `5432`, with a superuser role matching the current OS user (the macOS Homebrew default). The setup script will create the database `meowlytics_551`; no manual `CREATE DATABASE` is required.
- (Optional, for live AI analysis only) `GOOGLE_API_KEY` for Gemini 2.5 Flash. All Sec. 5 evidence runs without it.

A.2 One-command setup

From the repository root:

```
bash dsci551/setup.sh
```

This script:

1. Creates the `meowlytics_551` database if missing.
2. Writes a `.env` file pointing Prisma at the local instance.
3. Runs `npm install`.
4. Runs `npx prisma db push` to create the schema and indexes.
5. Runs the deterministic seed (`dsci551/seed/seed.ts`) — 10,000 ingredients, 51 users, 5,000 favorites, fixed Mulberry32 PRNG.
6. Runs `ANALYZE` so planner statistics match the values quoted in this report.
7. Executes a smoke `EXPLAIN ANALYZE` to confirm the install.

A.3 Reproduce the EXPLAIN evidence

To regenerate every plan in Sec. 5:

```
for f in dsci551/explain/*.sql; do
  echo "=== $f ==="
  psql -P pager=off meowlytics_551 -f "$f"
done
```

Files map to mappings as follows:

Script	Mapping
`01-ingredient-exact-lookup.sql`	Sec. 5.1
`02-favorites-composite-index.sql`	Sec. 5.2
`03-fuzzy-ilike-seq-scan.sql`	Sec. 5.5
`04-with-vs-without-index.sql`	Sec. 5.3
`05-small-vs-large-data.sql`	Sec. 5.4
`06-mvcc-snapshot-isolation.sql`	Sec. 5.7

A.4 Run the web app

```
npm run dev                          # http://localhost:3000
```

Demo login: `demo@551.edu` / `demo551`. The demo user already owns 2,000 seeded favorites for testing the favorites page.

A.5 Troubleshooting

- **`could not connect to server`** — PostgreSQL is not running. On macOS: `brew services start postgresql@16`.
- **`role "<user>" does not exist`** — create one matching your OS user: `createuser -s $(whoami)`.
- **Port 3000 already in use** — Next.js will pick 3001 automatically; or stop the conflicting process (`lsof -i :3000`).
- **Plan numbers don't match the report** — run `psql meowlytics_551 -c "ANALYZE;"` to refresh statistics.

Full setup notes: `dsci551/README.md`.

Appendix B: Repository Layout

```
dsci551-project-meowlytics/
├── app/                  Next.js application (UI + API routes)
├── lib/                  Domain logic (auth, knowledge search, validation)
├── prisma/schema.prisma  Database schema and indexes
├── dsci551/
│   ├── README.md         Setup, troubleshooting, mapping table
│   ├── setup.sh          One-command reproducer
│   ├── seed/seed.ts      Deterministic seed (10k + 51 + 5k rows)
│   ├── explain/          Six EXPLAIN ANALYZE evidence scripts
│   ├── slides/           Demo slide deck (PDF + PPTX)
│   └── docs/
│       └── FINAL_REPORT.md   ← this file (incl. Appendix C: Prisma schema)
└── .env                  Demo-safe local configuration
```

Appendix C: Prisma Schema

Complete schema referenced in Sec. 3.2. Indexes are declared via `@@index`.

```
model Ingredient {
  id              String   @id @default(cuid())
  name            String
  nameEn          String
  aliases         String[]
  category        String
  healthImpact    String
  description     String
  benefits        String[]
  concerns        String[]
  suitableFor     String[]
  notSuitableFor  String[]
  source          String

  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  @@index([name])
  @@index([nameEn])
  @@index([source])
}

model User {
  id           String     @id @default(cuid())
  email        String     @unique
  passwordHash String
  displayName  String?
  favorites    Favorite[]
  folders      Folder[]

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

model Favorite {
  id       String @id @default(cuid())
  userId   String
  user     User   @relation(fields: [userId], references: [id], onDelete: Cascade)
  name     String
  brand    String?
  imageData String?
  analysis Json
  notes    String?
  folderId  String?
  folder    Folder?  @relation(fields: [folderId], references: [id], onDelete: SetNull)

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  @@index([userId, createdAt])
}

model Folder {
  id        String     @id @default(cuid())
```

```
  userId    String
  user      User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  name      String
  color     String     @default("#f97316")
  favorites Favorite[]

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  @@index([userId, createdAt])
}
```